Chandigarh Engineering College Jhanjeri
Mohali-140307
Department of Computer Science & Engineering

PRACTICAL FILE

Department: Computer Science and Engineering

Subject: Parallel Computing

Subject Code: BTCS 714-18

INDEX

S NO.	NAME OF THE EXPERIMENTS	PAGE NO.	DATE	SIGN.
1.	Introduction to Parallel Computing			
2.	Demonstrate the concept of SIMD and MIMD			
3.	Explain PRAM Model			
4.	Basics of Message passing system			
5.	Design and implementation of various combinational circuit			
6.	Explain Various Performancee Metrics			
7.	Demonstrate benchmarks			
8.	Demonstrate the concept of Object Oriented Programming			
9.	Explain the concept of shared memory and distributed memory			
10.	Demonstrate the concept of parallel programming			

Practical - 1

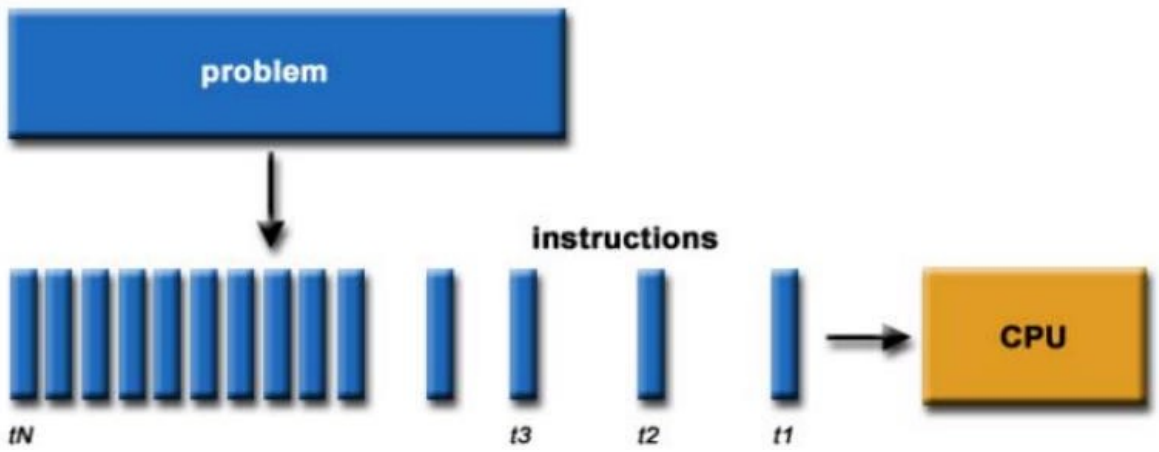
AIM : Introduction of Parallel Computing

Parallel Computing :

It is the use of multiple processing elements simultaneously for solving any problem. Problems are broken down into instructions and are solved concurrently as each resource that has been applied to work is working at the same time.

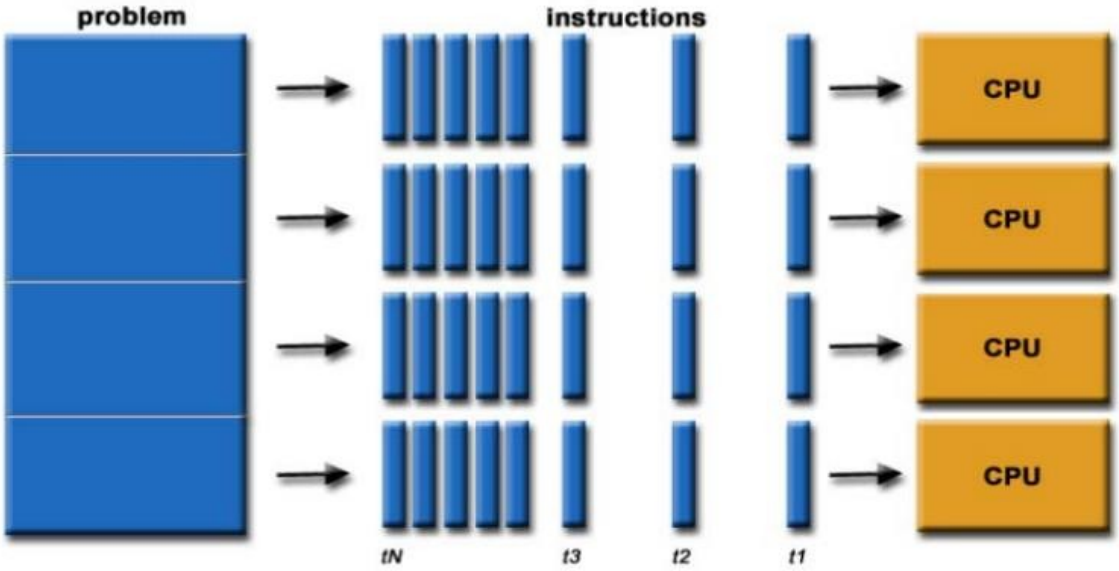
Traditionally software has been written for serial computations:

- To be run on a single computer having a single Central Processing Unit (CPU).
- A problem is broken into a discrete set of instructions.
- Instructions are executed one after another .
- Only one instruction can be executed at any moment in time.



In the simplest sense, parallel computing is the simultaneous use of multiple compute resources to solve a computational problem:

- To be run using multiple CPUs
- A problem is broken into discrete parts that can be solved concurrently.
- Each part is further broken down to a series of instructions.
- Instructions from each part execute simultaneously on different CPUs.



Why Use Parallel Computing?

- The whole real-world runs in dynamic nature i.e. many things happen at a certain time but at different places concurrently. This data is extensively huge to manage.
- Real-world data needs more dynamic simulation and modeling, and for achieving the same, parallel computing is the key.
- Parallel computing provides concurrency and saves time and money.
- Complex, large datasets, and their management can be organized only and only using parallel computing's approach.
- Ensures the effective utilization of the resources. The hardware is guaranteed to be used effectively whereas in serial computation only some part of the hardware was used and the rest rendered idle.
- Also, it is impractical to implement real-time systems using serial computing.

Major reasons:

Save time and/or money:

In theory, throwing more resources at a task will shorten its time to completion, with potential cost savings. Parallel clusters can be built from cheap, commodity components.

Solve larger problems:

Many problems are so large and/or complex that it is impractical or impossible to solve them on a single computer, especially given limited computer memory.

Provide concurrency:

A single compute resource can only do one thing at a time. Multiple computing resources can be doing many things simultaneously.

Use of non-local resources:

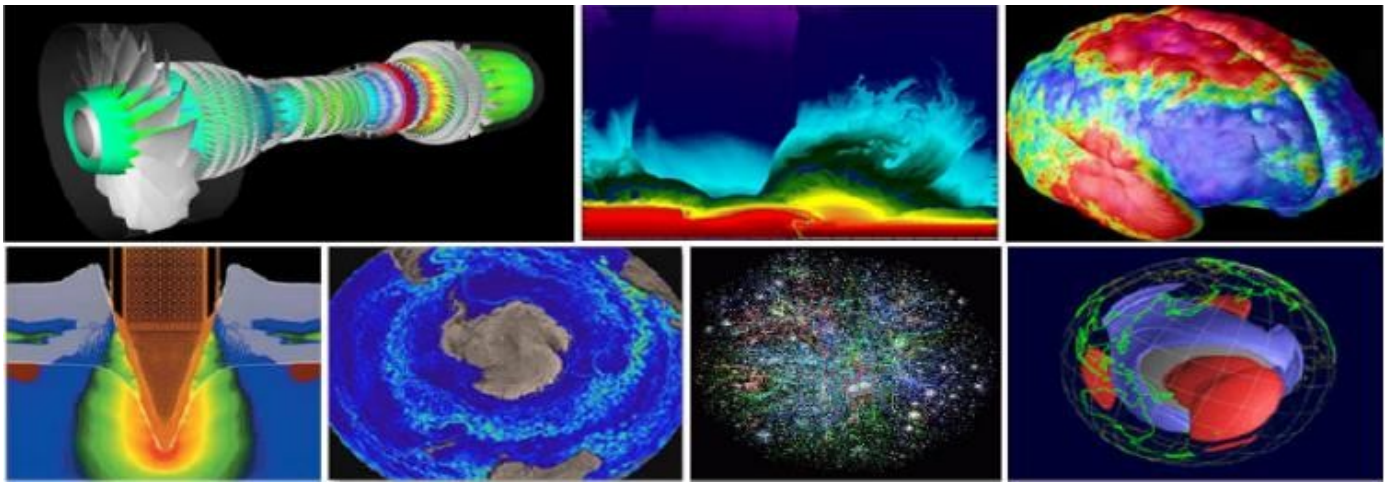
Using compute resources on a wide area network, or even the Internet when local compute resources are scarc.

Applications of Parallel Computing :

Historically, parallel computing has been considered to be "the high end of computing", and has been used to model difficult scientific and engineering problems found in the real world.

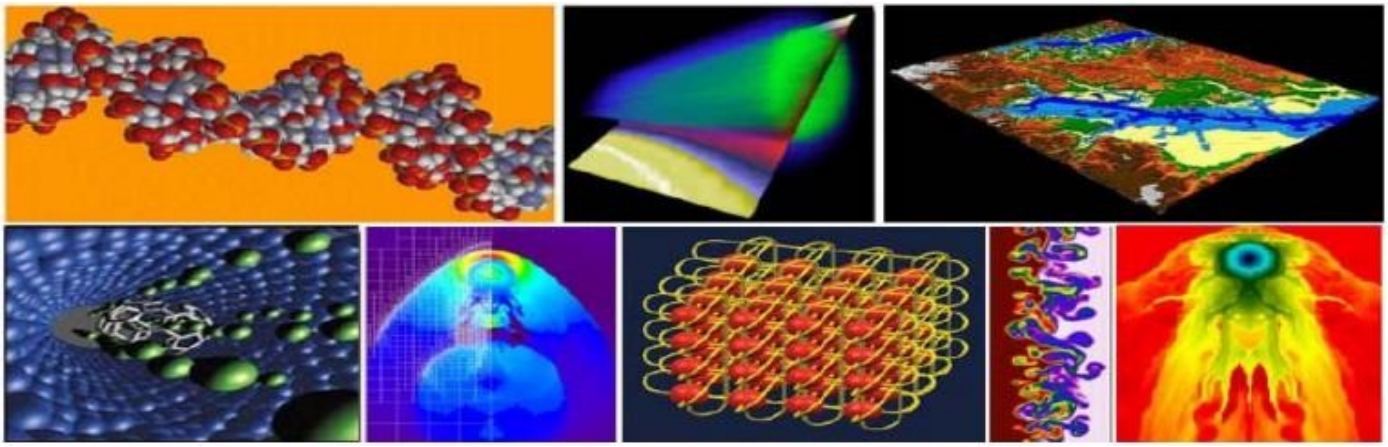
Some examples:

- Atmosphere, Earth, Environment, Space Weather.
- Physics -applied, nuclear, particle, condensed matter, high pressure,fusion, photonics.
- Bioscience, Biotechnology, Genetics
- Chemistry, Molecular Sciences
- Geology, Seismology
- Mechanical and Aerospace Engineering
- Electrical Engineering, Circuit Design, Microelectronics
- Computer Science, Mathematics



Today, commercial applications provide an equal or greater driving force in the development of faster computers. These applications require the processing of large amounts of data in sophisticated ways. For example:

- Databases, data mining
- Oil exploration
- Web search engines, web based business services
- Medical imaging and diagnosis
- Pharmaceutical design
- Management of national and multi-national corporations
- Financial and economic modeling
- Advanced graphics and virtual reality, particularly in the entertainment industry
- Networked video and multi-media technologies
- Collaborative work environments



Limitations of Parallel Computing:

- It addresses such as communication and synchronization between multiple sub-tasks and processes which is difficult to achieve.
- The algorithms must be managed in such a way that they can be handled in a parallel mechanism.
- The algorithms or programs must have low coupling and high cohesion. But it's difficult to create such programs.
- More technically skilled and expert programmers can code a parallelism-based program well.

Advantages of Parallel Computing over Serial Computing are as follows:

1. It saves time and money as many resources working together will reduce the time and cut potential costs.
2. It can be impractical to solve larger problems on Serial Computing.
3. It can take advantage of non-local resources when the local resources are finite.
4. Serial Computing 'wastes' the potential computing power, thus Parallel Computing makes better Work of the hardware.

Differnce between Parallel computing and Single computing

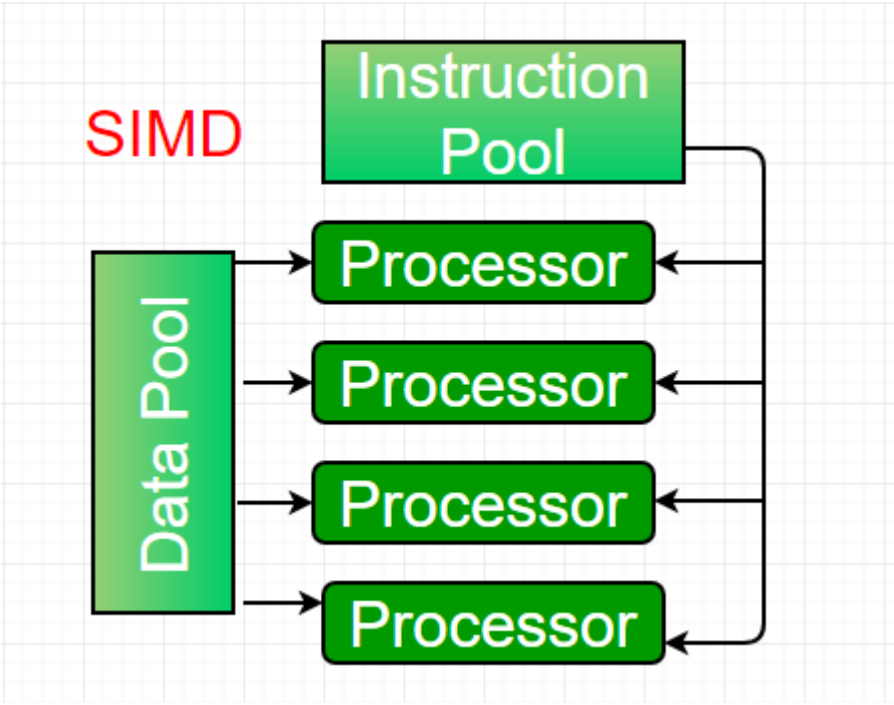
PARALLEL COMPUTING	**SINGLE COMPUTING**
Many operations are performed simultaneously.	System components are located at different locations.
Single computer is required.	Uses multiple computers.
Multiple processors perform multiple operations.	Multiple computers perform multiple operations.
It may have shared or distributed memory	It have only distributed memory
Processors communicate with each other through bus.	Computer communicate with each other through message passing.
Improves the system performance.	Improves system scalability, fault tolerance and resource sharing capabilities.

Practical - 2

AIM: Demonstrate the concept of SIMD and MIMD

Single-instruction, multiple-data (SIMD) systems –

An SIMD system is a multiprocessor machine capable of executing the same instruction on all the CPUs but operating on different data streams. Machines based on an SIMD model are well suited to scientific computing since they involve lots of vector and matrix operations. So that the information can be passed to all the processing elements (PEs) organized data elements of vectors can be divided into multiple sets(N-sets for N PE systems) and each PE can process one data set.



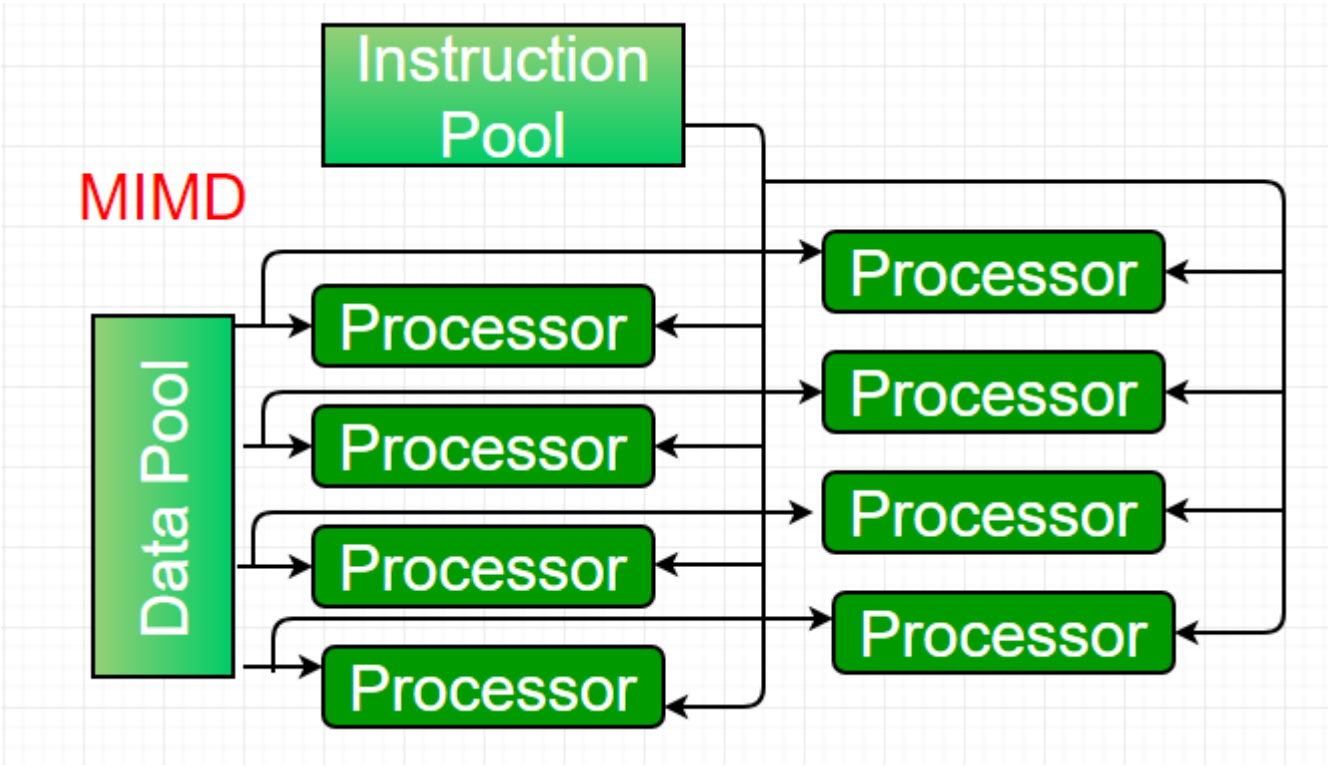
Dominant representative SIMD systems is Cray's vector processing machine.

Advantages of SIMD

- The creation, interpretation and debugging of the programs is done in an easier way due to single instruction stream and absolute synchronization of SIMD.
- Provides effective inter-process communication by employing implicit synchronization. It uses "send" and "receive" commands to acknowledge the receiver who send it, why it was sent and when to read it, rather than identification protocols.
- It can overlap control flow instructions and multiple scalar operations over the CU.
- Requires less memory as the single copy of the instruction is stored in the system memory.
- The single instruction decoder is needed which reduces SIMD overall cost.

Multiple-instruction, multiple-data (MIMD) systems –

An MIMD system is a multiprocessor machine which is capable of executing multiple instructions on multiple data sets. Each PE in the MIMD model has separate instruction and data streams; therefore machines built using this model are capable to any kind of application. Unlike SIMD and MISD machines, PEs in MIMD machines work asynchronously.



MIMD machines are broadly categorized into shared-memory MIMD and distributed-memory MIMD based on the way PEs are coupled to the main memory.

In the shared memory MIMD model (tightly coupled multiprocessor systems), all the PEs are connected to a single global memory and they all have access to it. The communication between PEs in this model takes place through the shared memory, modification of the data stored in the global memory by one PE is visible to all other PEs. Dominant representative shared memory MIMD systems are Silicon Graphics machines and Sun/IBM's SMP (Symmetric Multi-Processing).

In Distributed memory MIMD machines (loosely coupled multiprocessor systems) all PEs have a local memory. The communication between PEs in this model takes place through the interconnection network (the inter process communication channel, or IPC). The network connecting PEs can be configured to tree, mesh or in accordance with the requirement.

The shared-memory MIMD architecture is easier to program but is less tolerant to failures and harder to

extend with respect to the distributed memory MIMD model. Failures in a shared-memory MIMD affect the entire system, whereas this is not the case of the distributed model, in which each of the PEs can be easily isolated. Moreover, shared memory MIMD architectures are less likely to scale because the addition of more PEs leads to memory contention. This is a situation that does not happen in the case of distributed memory, in which each PE has its own memory. As a result of practical outcomes and user's requirement , distributed memory MIMD architecture is superior to the other existing models.

Advantages of MIMD

- Provides high flexibility by employing multiple threads of control.
- Facilitates efficient execution of the conditional statements (i.e., if-then-else) because the processor is independent and can follow any random decision path.
- The asynchronous property of MIMD can speed up the execution rate of instructions which take an indefinite amount of time.
- Does not require additional CU and hence do not have supplementary cost.

Differences Between SIMD and MIMD

SIMD	**MIMD**
1. SIMD stands for Single Instruction Multiple Data.	While MIMD stands for Multiple Instruction Multiple Data.
2. SIMD requires small or less memory.	While it requires more or large memory.
3. The cost of SIMD is less than MIMD.	While it is costlier than SIMD.
4. It has single decoder.	While it have multiple decoders.
5. It is latent or tacit synchronization.	While it is accurate or explicit synchronization.
6. SIMD is a synchronous programming.	While MIMD is a asynchronous programming.
7. SIMD is a simple in terms of complexity	While MIMD is complex in terms of complexity

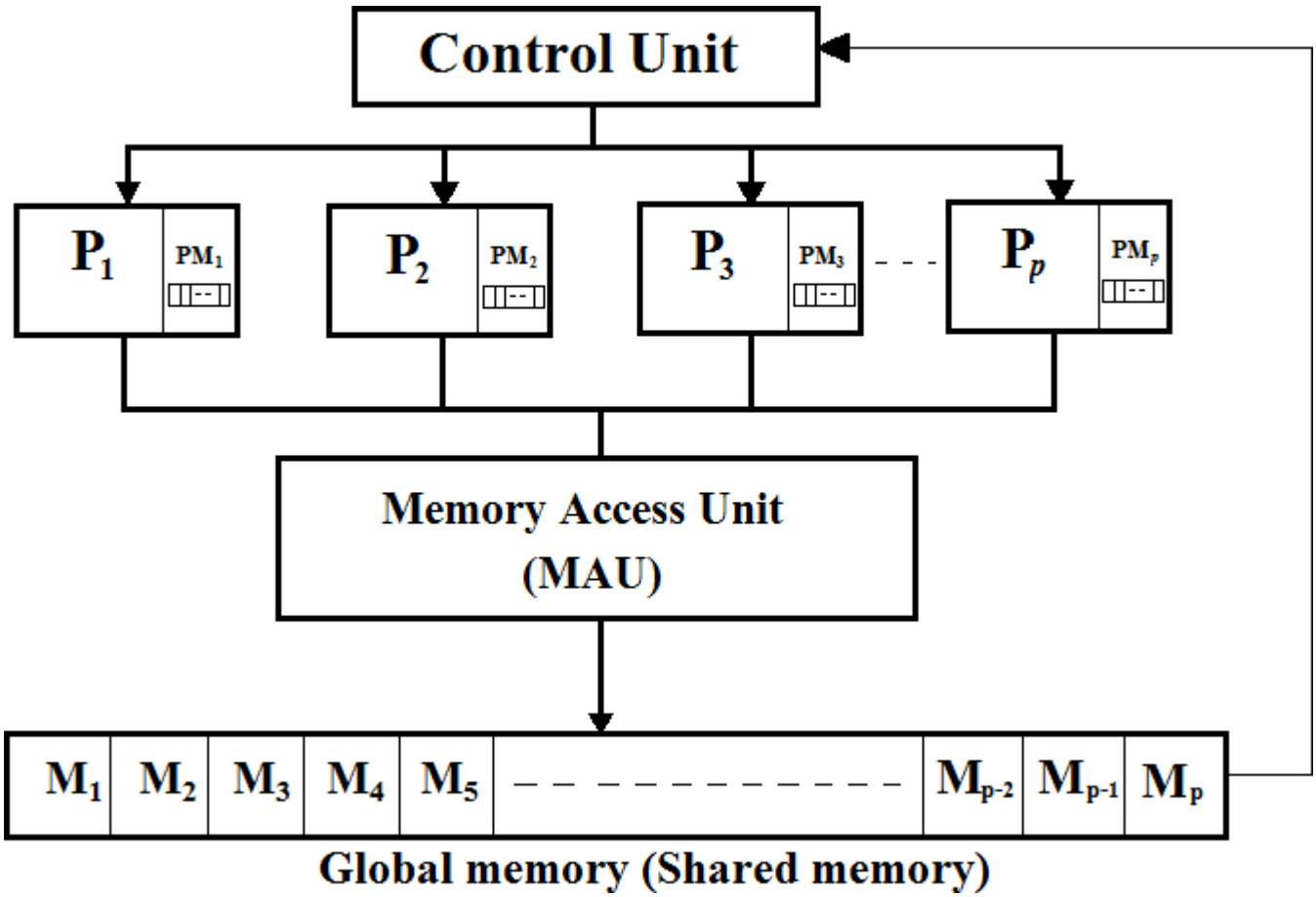
than MIMD.	than SIMD.
8. SIMD is less efficient in terms of performance than MIMD.	While MIMD is more efficient in terms of performance than SIMD.

PRACTICAL - 3

AIM : Explain PRAM Model

Parallel Random Access Machine, also called as PRAM, is a model considered for most of the parallel algorithms. It is the extension of RAM model for sequential algorithm.

It helps to write a precursor parallel algorithm without any architecture constraints and also allows parallel-algorithm designers to treat processing power as unlimited. It ignores the complexity of inter-process communication. PRAM algorithms are mostly theoretical but can be used as a basis for developing an efficient parallel algorithm for practical machines and can also motivate building specialized machines.



In PRAM model multiple processors are attached to a single block of memory called global memory. In the above block diagram of PRAM model of parallel computation, p number of processors can perform

independent operations on p number of data in a particular unit of time. This may result in simultaneous access of same memory location by different processors. A PRAM model consist the following−

- PRAM model consist a finite set of similar type of processors. Every processor has a unique index which is used to enable or disable the processor. This unique index number identifies which memory location accesses by a processor. These unbounded set of processors have their own private memory for local data store.
- PRAM model consist a common memory unit called global memory or shared memory. Every processor in PRAM shares this common memory unit. Processors can communicate among themselves through the shared memory only.
- PRAM model consist of a control unit to control the instruction fetched by all processors in PRAM model.
- A memory access unit (MAU) is connects the processors with the single shared memory.
- PRAM processors have a read, compute and write phases that happen synchronously.

Models of PRAM: While accessing the shared memory, there can be conflicts while performing the read and write operation (i.e.), a processor can access a memory block which is already being accessed by another processor. Therefore, there are various constraints on a PRAM model which handle the read or write conflicts. They are:

EREW: also called Exclusive Read Exclusive Write is a constraint which doesn't allow two processors to read or write from the same memory location at the same instance.

CREW: also called Concurrent Read Exclusive Write is a constraint which allows all the processors to read from the same memory location but are not allowed to write into the same memory location at the same time.

ERCW: also called Exclusive Read Concurrent Write is a constraint which allows all the processors to write to the same memory location but are now allowed to read the same memory location at the same time.

CRCW: also called Concurrent Read Concurrent Write is a constraint which allows all the processors to read from and write to the same memory location parallelly.

PRAM instruction execute in three phase cycle:

- Read instruction execution:
- Local computation:
- Write instruction execution:

Steps involved in PRAM computation:

- A PRAM computation begins with a single active processing elements and the input stored in global memory.
- During each step of computation an active enabled process may read a value from single private or global memory location. Perform a single RAM operation and write into one local or global memory location.
- During computation step processors have a special activation register that specifying the maximum index of an active processor. Initially, only the first processor is in active mode, it computes the number of required active processors and loads this register, and then the other corresponding processors start executing their programs. The computation will proceeds until the first processor halts, at which time all other active processors are halted.

Features of PRAM model:

To design parallel algorithm PRAM is an attractive and important model because of its features as follows:

- PRAM model is natural: The number of operations executed per one cycle on p processors is at most p.
- PRAM model is strong: any processor can read or write any shared memory cell in unit time.
- PRAM model is simple: It abstracts from any communication or synchronization overhead, which makes the complexity and correctness analysis of PRAM algorithms easier. Therefore, It can be used as a benchmark: If a problem has no feasible/efficient solution on PRAM, it has no feasible/efficient solution on any parallel machine.
- PRAM model is useful: It is an idealization of existing (and nowaday more and more abundant) shared memory parallel machines.

Constrained on PRAM Model:

Bounded number of shared memory cells: This may be called a small memory PRAM. If the input data set or set of instruction exceeds the capacity of the shared memory then the input and/or output values of shared memory cells can be distributed evenly among the processors.

Bounded size of a machine word and/or memory cell: The parameter of memory cell in PRAM model is presenting the size of a machine word.

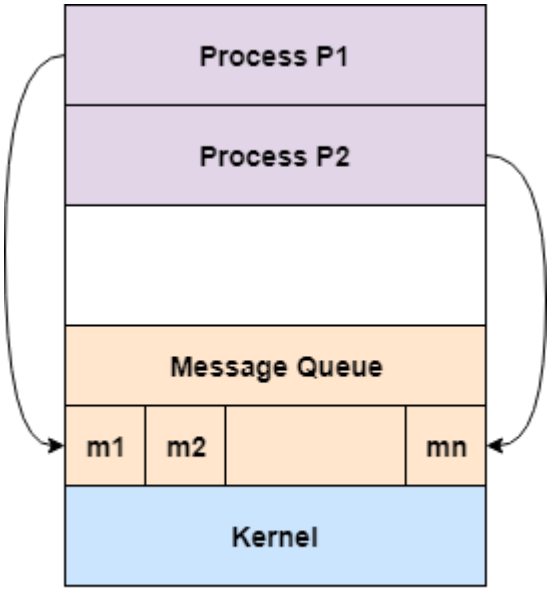
Bounded number of processors: If the number of threads of execution is higher, processors may interleave several threads sometime it named as a small PRAM.

Constraints on simultaneous access to shared memory cells: It is based on handling the memory access conflicts.

PRACTICAL - 4

AIM : Basics of Message Passing Interface

Message passing model allows multiple processes to read and write data to the message queue without being connected to each other. Messages are stored on the queue until their recipient retrieves them. Message queues are quite useful for interprocess communication and are used by most operating systems.



Message Passing Model

Advantages of Message Passing Model

Some of the advantages of message passing model are given as follows −

- The message passing model is much easier to implement than the shared memory model.
- It is easier to build parallel hardware using message passing model as it is quite tolerant of higher communication latencies.

Disadvantage of Message Passing Model

The message passing model has slower communication than the shared memory model because the connection setup takes time.

The **Message Passing Interface** or MPI is a standard for message passing that has been developed by a consortium consisting of representatives from research laboratories, universities, and industry.

- The first version MPI-l was standardized in 1994, and the second version MPI-2 was developed in 1997. MPI is an explicit message passing paradigm where tasks communicate with each other by sending messages.
- The two main objectives of MPI are portability and high performance. The MPI environment consists of an MPI library that provides a rich set of functions numbering in the hundreds. MPI defines the concept of communicators which combine message context and task group to provide message security. Intra-communicators allow safe message passing within a group of tasks, and intercommunicates allow safe message passing between two groups of tasks.
- MPI provides many different flavors of both blocking and non-blocking point to point communication primitives, and has support for structured buffers and derived data types. It also provides many different types of collective communication routines for communication, between tasks belonging to a group. Other functions include those for application-oriented task topologies, profiling, and environmental query and control functions. MPI-2 also adds dynamic spawning of MPI tasks to this impressive list of functions.

Key Points:

- MPI is a library, not a language.
- MPI is a specification , not a particular implementation
- MPI addresses the message passing model.

Implementation of MPI: MPICH

MPICH is one of the complete implementation of the MPI specification, designed to be both portable and efficient. The ``CH" in MPICH stands for ``Chameleon," symbol of adaptability to one's environment and thus of portability. Chameleons are fast, and from the beginning a secondary goal was to give up as little efficiency as possible for the portability. MPICH is a unified source distribution, supporting most flavors of Unix and recent versions of Windows. In additional, binary distributions are available for Windows platforms.

Structure of MPI Program:

```
#include <mpi.h>

int main(int argc, char ** argv)

//Serial Code
```

```
{
    MPI_Init(&argc,&argv);
    //Parallel Code
    MPI_Finalize();
    //Serial Code
}
```

A simple MPI program contains a main program in which parallel code of program is placed between PI_Init and MPI_Finalize.

- **MPI_Init**

It is used initializes the parallel code segment. Always use to declare the start of the parallel code segment.

```
int MPI_Init( int* argc ptr /* in/out */ ,char** argv ptr[ ] /* in/out */)
    or simply
MPI_Init(&argc,&argv)
```

- **MPI Finalize**

It is used to declare the end of the parallel code segment. It is important to note that it takes no arguments.

```
int MPI Finalize(void)
or simply
MPI_Finalize()
```

Key Points:

- Must include mpi.h by introducing its header #include<mpi.h>. This provides us with the function declarations for all MPI functions.

- A program must have a beginning and an ending. The beginning is in the form of an MPI_Init() call, which indicates to the operating system that this is an MPI program and allows the OS to do any necessary initialization. The ending is in the form of an MPI_Finalize() call, which indicates to the OS that "clean-up" with respect to MPI can commence.
- If the program is embarrassingly parallel, then the operations done between the MPI initialization and finalization involve no communication.

Predefined Variable Types in MPI

MPI DATA TYPE	***C DATA TYPE***
MPI_CHAR	Signed Char
MPI_SHORT	Singed Short Int
(Cont.)	
MPI_INT	Signed Int
MPI_LONG	Singed Long Int
MPI_UNSIGNED_CHAR	Unsigned Char
MPI_UNSIGNED_SHORT	Unsigned Short Int
MPI_UNSIGNED	Unsigned Int
MPI_UNSIGNED_LONG	Unsigned Long Int
MPI_FLOAT	Float
MPI_DOUBLE	Double
MPI_LONG_DOUBLE	Long Double
MPI_BYTE	----------
MPI_PACKED	----------

PRACTICAL - 5

AIM : Design and implementation of various combinational circuit

Combinational circuit is a circuit in which we combine the different gates in the circuit, for example encoder, decoder, multiplexer and demultiplexer. Some of the characteristics of combinational circuits are following –

- The output of combinational circuit at any instant of time, depends only on the levels present at input terminals.
- The combinational circuit do not use any memory. The previous state of input does not have any effect on the present state of the circuit.
- A combinational circuit can have an n number of inputs and m number of outputs.

Block diagram

LEARNING OBJECTIVE :

- To design, realize and verify the adder and subtractor circuits using basic gates and universal gates.
- To design, realize and verify full adder using two half adders.
- To design, realize and verify a full subtractor using two half subtractors.

COMPONENTS REQUIRED: IC 7400, IC 7408, IC 7486, AND IC 7432, Patch cards and IC Trainer Kit.

THEORY :

Half Adder : A combinational logic circuit that performs the addition of two data bits. A and B, is called a half adder. Addition will result in two output bits; one of which is the sum bit, S, and the other is the carry bit, C. the boolean functions describing the half adder are:

$$S = A \oplus B \qquad C = A\,B$$

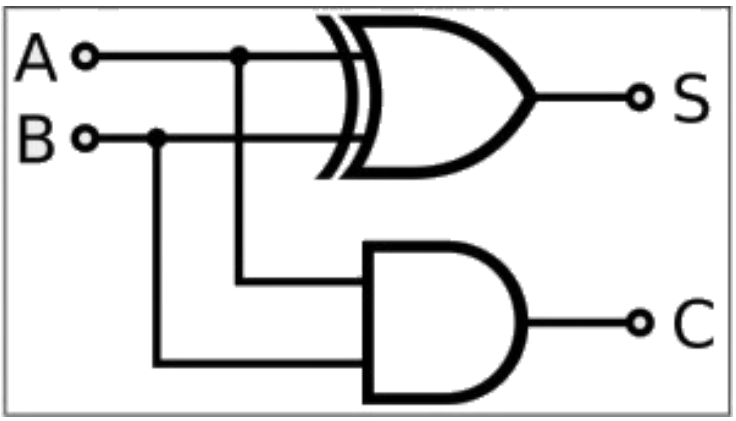
To realize Half -adder

Truth Table			
Input		Output	
A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

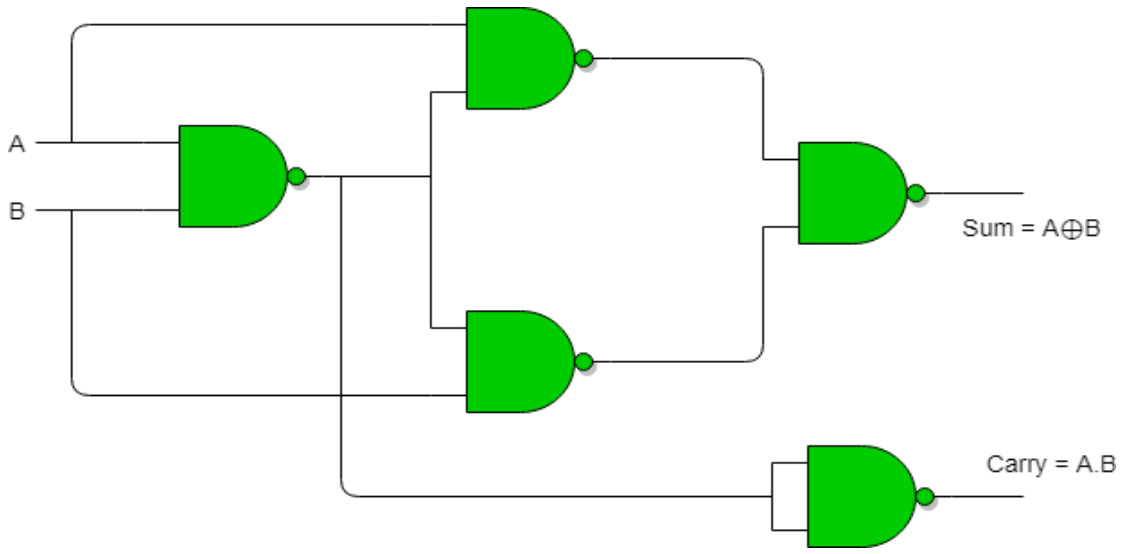
BOOLEAN EXPRESSIONS :

S = A ⊕B **C = A B**

BASIC GATES



NAND GATES



FULL ADDER

The half adder does not take the carry bit from its previous stage into account. This carry bit from its previous stage is called carry- in bit. A combinational logic circuit that adds two data bits, A and B, and a carry-in bit, cin, is called a full adder. The boolean functions describing the full adder are:

S= (x ⊕y) ⊕Cin **C = xy + Cin (x ⊕y)**

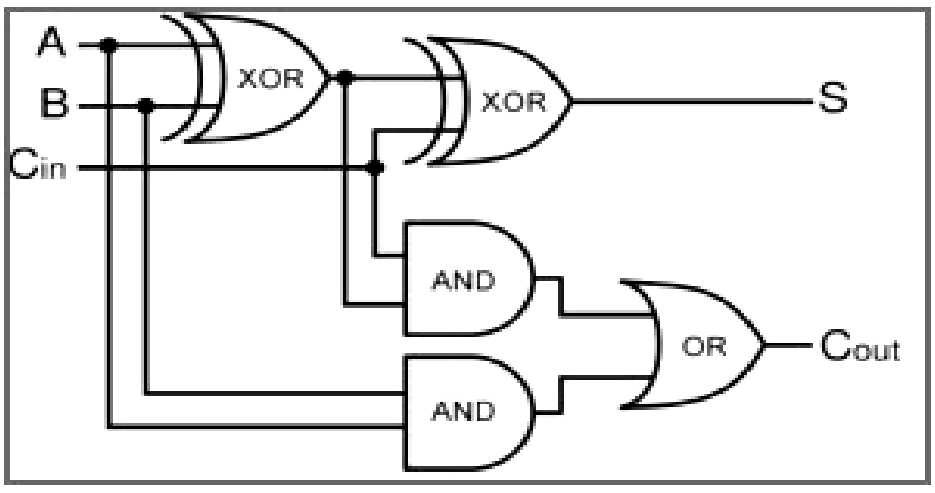
TRUTH TABLE

Input			Output	
A	B	Cin	Sum	Carry
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

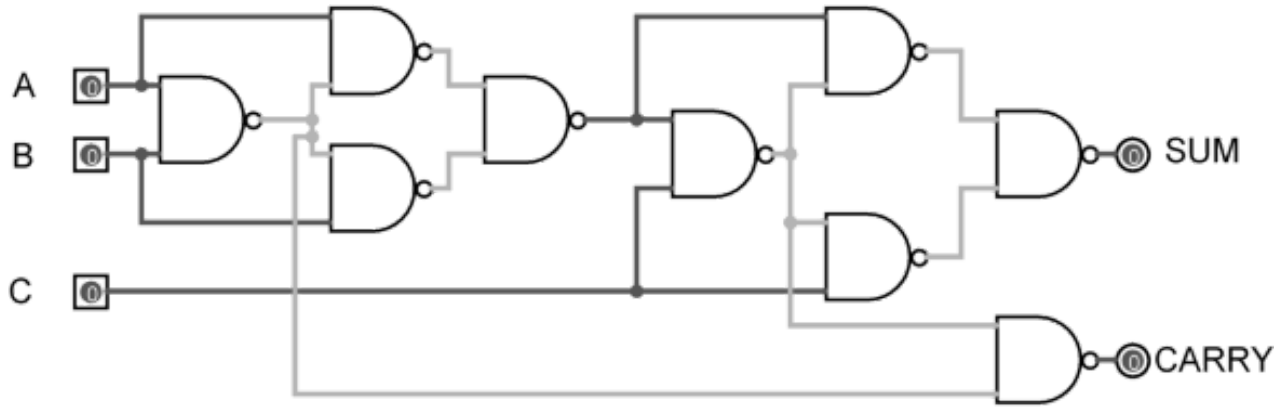
BOOLEAN EXPRESSIONS :

S = A ⊕B ⊕C **C = A B + B Cin + A Cin**

BASIC GATES



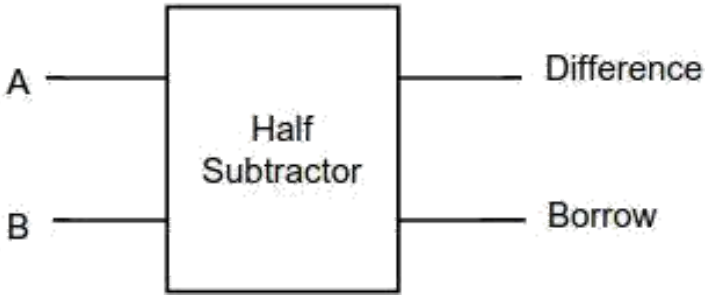
NAND GATES



HALF SUBTRACTOR

The half-subtractor is a combinational circuit which is used to perform subtraction of two bits. It has two inputs, A (minuend) and B (subtrahend) and two outputs Difference and Borrow.

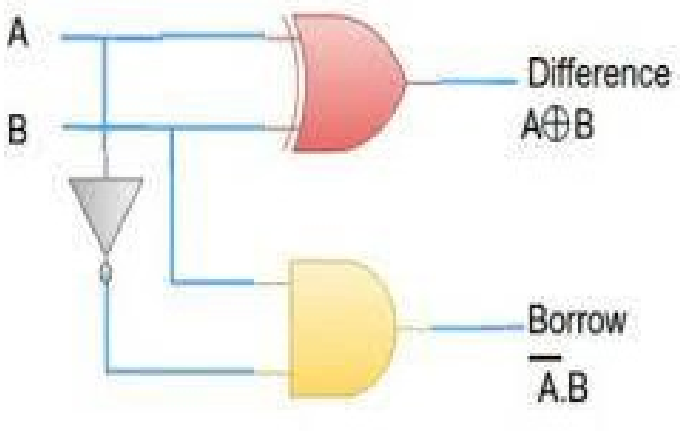
Logic Symbol of Half subtractor



Truth Table of Half subtractor

Inputs		Outputs	
A	B	Difference	Borrow
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

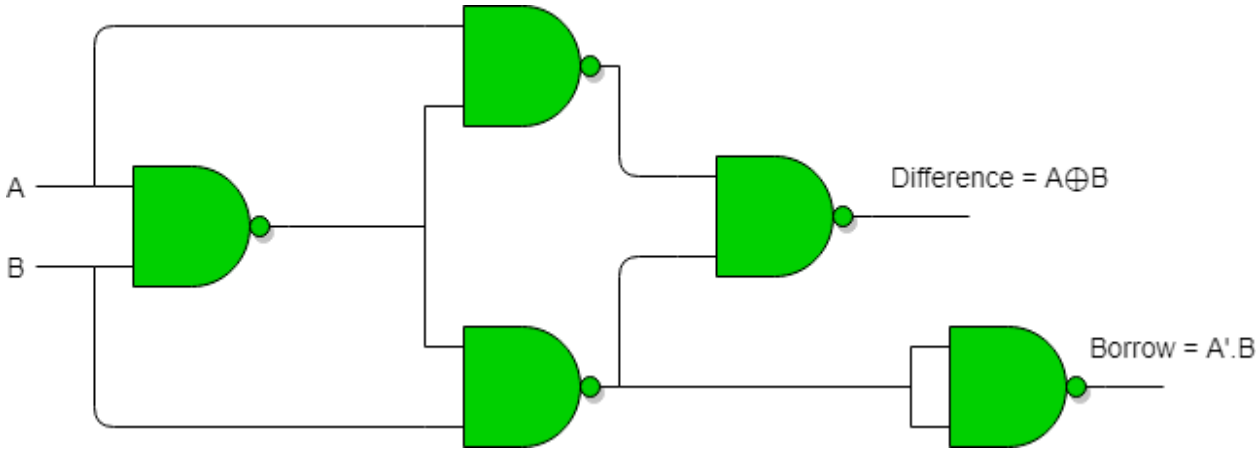
Circuit Diagram of Half subtractor



From the above truth table we can find the boolean expression.

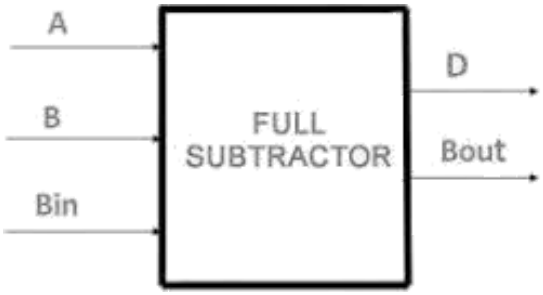
Difference = A ⊕B **Borrow = A' B**

NAND Gates :



FULL SUBTRACTOR

A full subtractor is a combinational circuit that performs subtraction involving three bits, namely A (minuend), B (subtrahend), and Bin (borrow-in) . It accepts three inputs: A (minuend), B (subtrahend) and a Bin (borrow bit) and it produces two outputs: D (difference) and Bout (borrow out). The logic symbol and truth table are shown below.



Truth Table of Full subtractor

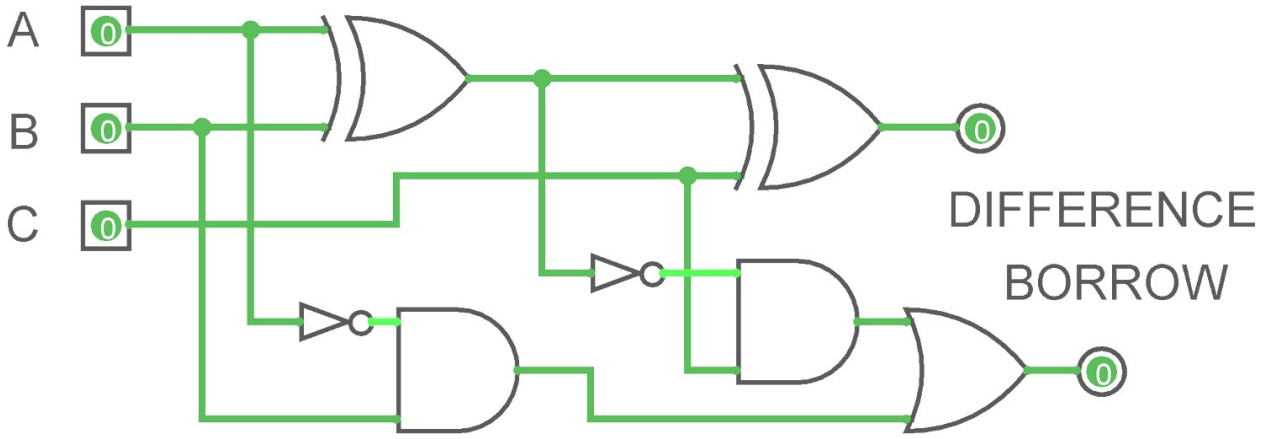
A	B	B_{in}	D	B_{out}
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

From the above truth table we can find the boolean expression.

D = A ⊕B ⊕Bin

Bout = A' Bin + A' B + B Bin

Circuit Diagram of Full subtractor


A
B
C
0
0
0
0
0
DIFFERENCE
BORROW

PRACTICAL - 6

AIM : Explain Various Performance Metrics

Performance Metrices

- **Run Time:** The parallel run time is defined as the time that elapses from the moment that a parallel computation starts to the moment that the last processor finishes execution.
- **Notation:** Serial run time, parallel run time .

We need performance matrices so that the performance of different processors can be measured and compared. The performance of a processor majorly depends on the clock speed and it is mentioned by the manufacturers.

There are 2 distinct classes of performance metrics:

- **Performance metrics for processors/cores –** assess the performance of a processing unit, normally done by measuring the speed or the number of operations that it does in a certain period of time
- **Performance metrics for parallel applications** – assess the performance of a parallel application, normally done by comparing the execution time with multiple processing units against the execution time with just one processing R. Rocha and F. Silva (DCC-FCUP) Performance Metrics Parallel Computing 15/16 4 multiple processing units against the execution time with just one processing unit.

Some Performance Metrics :

Speedup

Measures performance gain achieved by parallelizing a given application over sequential implementation

– Captures relative benefit of solving a problem in parallel

- Defined as ratio of time taken to solve a problem on a single PE to time required to solve the same problem on a parallel computer with p PEs, that is

S = Ts/Tp

Speedup is a measure of performance. It measures the ratio between the sequential execution time and the parallel execution time. ( ))1(( ) T p T S p = R. Rocha and F. Silva (DCC-FCUP) Performance Metrics Parallel Computing 15/16 7 T(1) is the execution time with one processing unit T(p) is the execution time with p processing units

	1 CPU	2 CPUs	4 CPUs	8 CPUs	16 CPUs
T(p)	1000	520	280	160	100
S(p)	1	1.92	3.57	6.25	10.00

Efficiency is a measure of the usage of the computational capacity. It measures the ratio between performance and the number of resources available to achieve that performance. ( ) ( ))1(( ) p T p T p S p E p × = = R. Rocha and F. Silva (DCC-FCUP) Performance Metrics Parallel Computing 15/16 8 S(p) is the speedup for p processing units

	1 CPU	2 CPUs	4 CPUs	8 CPUs	16 CPUs
S(p)	1	1.92	3.57	6.25	10.00
E(p)	1	0.96	0.89	0.78	0.63

- Measures fraction of time for which a PE is usefully employed
- Defined as ratio of the speedup to the number of PEs

  E = S/p = TS / (TP * p)
- In ideal parallel systems
  - Speedup is equal to p, and
  - Efficiency is equal to one
- In practice
  - Speedup is less than p, and
  - Efficiency is between zero and one

Redundancy

Redundancy measures the increase in the required computation when using more processing units. It measures the ratio between the number of operations performed by the parallel execution and by the sequential execution. (1) ( ) ( ) OO p R p = R. Rocha and F. Silva (DCC-FCUP) Performance Metrics Parallel Computing 15/16 9 O(1) is the total number of operations performed by one processing unit O(p) is the total number of operations performed by p processing units.

	1 CPU	2 CPUs	4 CPUs	8 CPUs	16 CPUs
O(p)	10000	10250	11000	12250	15000
R(p)	1	1.03	1.10	1.23	1.50

Utilization

Utilization is a measure of the good use of the computational capacity. It measures the ratio between the computational capacity used during parallel execution and the capacity that was available.

$U(p) = R(p) \times E(p)$

	1 CPU	2 CPUs	4 CPUs	8 CPUs	16 CPUs
R(p)	1	1.03	1.10	1.23	1.50
E(p)	1	0.96	0.89	0.78	0.63
U(p)	1	0.99	0.98	0.96	0.95

Scalability

- The efficiency of a parallel program can be written as or

$$E = \frac{S}{p} = \frac{T_S}{pT_P}$$

Or

$$E = \frac{1}{1 + \frac{T_o}{T_S}}.$$

- The total overhead function To is an increasing function of p
- For a given problem size
  - Value of Ts remains constant
  - As we increase number of PEs, To increases
  - The overall efficiency of the parallel program goes down
- This is the case for all parallel programs

PRACTICAL - 7

AIM : Demonstrate benchmarks

Benchmark: A standardized problem or test that serves as a basis for evaluation or comparison (as of computer system performance. The term "benchmark" also commonly applies to specially-designed programs used in benchmarking.

A benchmark should:

- be domain specific (the more general the benchmark, the less useful it is for anything in particular)
- be a distillation of the essential attributes of a workload
- avoid using single metric to express the overall performance
- Computational benchmark kinds

Synthetic: specially-created programs that impose the load on the specific component in the system

application: derived from a real-world application program.

Purpose of Benchmarking

1. Provide a tool, enabling quantitative comparisons
    - Comparison of variations within the same system
    - Comparison of distinct systems
2. Driving progress
    - Enable better engineering by defining measurable and repeatable objectives
3. Establishing of performance agenda
    - Measure release-to-release or version-to-version progress
    - Set goals to meet
    - Be understandable and useful also to the people not having the expertise in the field (managers, etc.)

Properties of a Good Benchmark

- Relevance: meaningful within the target domain
- Understandability
- Good metric(s): linear, orthogonal, monotonic
- Scalability: applicable to a broad spectrum of hardware/architecture
- Coverage: does not over-constrain the typical environment (does not require any special conditions)
- Acceptance: embraced by users and vendors

- Has to enable comparative evaluation

Early Benchmarks

- Whetstone
  - Floating point intensive
- Dhrystone
  - Integer and character string oriented
- Livermore Fortran Kernels
  - "Livermore Loops"
  - Collection of short kernels
- NAS kernel
  - 7 Fortran test kernels for aerospace computation

1. Whetstone

- Originally written in Algol 60 in 1972 at the National Physics Laboratory (UK)
- Named after Whetstone Algol translator-interpreter on the KDF9 computer
- Measures primarily floating point performance in WIPS: Whetstone Instructions Per Second
- Raised also the issue of efficiency of different programming languages
- The original Algol code was translated to C and Fortran (single and double precision support), PL/I, APL, Pascal, Basic, Simula and others

2. Dhrystone

- Synthetic benchmark developed in 1984 by Reinhold Weicker
- The name is a pun on "Whetstone"
- Measures integer and string operations performance, expressed in number of iterations, or Dhrystones, per second
- Alternative unit: D-MIPS, normalized to VAX 11/780 performance
- Latest version released: 2.1, includes implementations in C, Ada and Pascal
- Superseded by SPECint suite

3. Livermore Fortran Kernels (LFK)

- Developed at Lawrence Livermore National Laboratory in 1970
- Consists of 24 separate kernels:
- hydrodynamic codes, Cholesky conjugate gradient, linear algebra, equation of state, integration, predictors, first sum and difference, particle in cell, Monte Carlo, linear recurrence, discrete ordinate transport, Planckian distribution and others
- Include careful and careless coding practices

- Produces 72 timing results using 3 different DO-loop lengths for each kernel
- Produces Megaflops values for each kernel and range statistics of the results
- Can be used as performance, compiler accuracy (checksums stored in code) or hardware endurance test

4. NAS Kernel

- Developed at the Numerical Aerodynamic Simulation Projects Office at NASA Ames
- Focuses on vector floating point performance
- Consists of 7 test kernels in Fortran (approx. 1000 lines of code):
  - matrix multiply
  - complex 2-D FFT
  - Cholesky decomposition
  - block tri-diagonal matrix solver
  - vortex method setup with Gaussian elimination
  - vortex creation with boundary conditions
  - parallel inverse of three matrix pentadiagonals
- Reports performance in Mflops (64-bit precision)

Some examples of benchmarking?

Different industries use benchmarking in a variety of ways. Here are some examples of ways companies can use benchmarking to achieve specific results:

Call center

A call center might benchmark its customer satisfaction rating by asking customers to rate their service based on their experiences. They might also collect data about waiting times, call lengths, first contact resolution rating, occupancy and shrinkage. These figures could be used to boost performance by improving processes and systems and also as a tool to help improve motivation among the staff.

Technology

A technology company may monitor the specifications of their competitors' products and compare them to their own, or measure their product life cycle against industry averages to ensure that they stay competitive.

Healthcare

Medical settings often collect benchmarking data about their patients, including assessing waiting times, the quality of care, recovery times and patient satisfaction. These metrics can be collected internally to establish the rate of progress in each area, and the results can also be compared to those of similar organizations to assess their position in the wider landscape.

Hospitality

Benchmarking is key to staying competitive in the hospitality industry where everything is recorded and compared, from details of bar consumables and food costs to employee benefits and retention rates. Bars, diners, restaurants and hotels also use customer satisfaction ratings for benchmarking purposes to ensure that their staff training is effective and check that their processes are robust.

PRACTICAL - 8

AIM: Demonstrate the concept of Object Oriented Programming

Object oriented programming is a type of programming which uses objects and classes its functioning. The object oriented programming is based on real world entities like inheritance, polymorphism, data hiding, etc. It aims at binding together data and function work on these data sets into a single entity to restrict their usage.

Some basic concepts of object oriented programming are −

CLASS

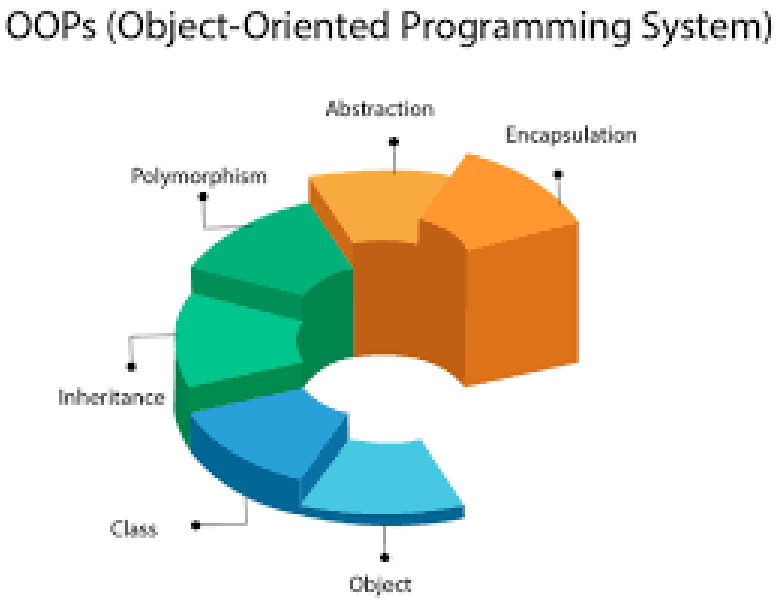
OBJECTS

ENCAPSULATION

POLYMORPHISM

INHERITANCE

ABSTRACTION



Class − A class is a data-type that has its own members i.e. data members and member functions. It is the blueprint for an object in object oriented programming language. It is the basic building block of object oriented programming in c++. The members of a class are accessed in programming language by creating an instance of the class.

Some important properties of class are −

- Class is a user-defined data-type.

- A class contains members like data members and member functions.
- Data members are variables of the class.
- Member functions are the methods that are used to manipulate data members.
- Data members define the properties of the class whereas the member functions define the behaviour of the class.
- A class can have multiple objects which have properties and behaviour that in common for all of them.

Syntax

```
class class_name {
  data_type data_name;
  return_type method_name(parameters);
}
```

Object − An object is an instance of a class. It is an entity with characteristics and behaviour that are used in the object oriented programming. An object is the entity that is created to allocate memory. A class when defined does not have memory chunk itself which will be allocated as soon as objects are created.

Syntax

```
class_name object_name;
```

Encapsulation In object oriented programming, encapsulation is the concept of wrapping together of data and information in a single unit. A formale defination of encapsulation would be: encapsulation is binding togather the data and related function that can manipulate the data.

Let’s understand the topic with an easy real life example,

In our colleges, we have departments for each course like computer science, information tech. , electronics, etc. each of these departments have their own students and subjects that are kept track of and being taught. let's think of each department as a class that encapsulates the data about students of that department and the subjects that are to be taught. Also a department has some fixed rules and guidelines that are to be followed by the students that course like timings, methods used while learning, etc. this is encapsulation in real life, there are data and there are ways to manipulate data.

Due to the concept of encapsulation in object oriented programming another very important concept is possible, it is data abstraction or Data Hiding. it is possible as encapsulating hides the data at show only the information that is required to be displayed.

Polymorphism The name defines polymorphism is multiple forms. which means polymorphism is the ability of object oriented programming to do some work using multiple forms. The behaviour of the method is dependent on the type or the situation in which the method is called.

Inheritance it is the capability of a class to inherit or derive properties or characteristics other class. it is very important and object oriented program as it allows reusability i.e. using a method defined in another class by using inheritance. The class that derives properties from other class is known as child class or subclass and the class from which the properties are inherited is base class or parent class.

C plus plus programming language supports the following types of inheritance

- single inheritance
- multiple inheritance
- multi level inheritance
- Hierarchical inheritance
- hybrid inheritance

Abstraction Data abstraction or Data Hiding is the concept of hiding data and showing only relevant data to the final user. It is also an important part object oriented programing.

let's take real life example to understand concept better, when we ride a bike we only know that pressing the brake will stop the bike and rotating the throttle will accelerate but you don't know how it works and it is also not think we should know that's why this is done from the same as a concept data abstraction.

What are the benefits of OOP?

Benefits of OOP include:

- **Modularity.** Encapsulation enables objects to be self-contained, making troubleshooting and collaborative development easier.
- **Reusability.** Code can be reused through inheritance, meaning a team does not have to write the same code multiple times.
- **Productivity.** Programmers can construct new programs quicker through the use of multiple libraries and reusable code.
- **Easily upgradable and scalable**. Programmers can implement system functionalities independently. Interface descriptions. Descriptions of external systems are simple, due to message passing techniques that are used for objects communication.

- **Security.** Using encapsulation and abstraction, complex code is hidden, software maintenance is easier and internet protocols are protected.
- **Flexibility.** Polymorphism enables a single function to adapt to the class it is placed in. Different objects can also pass through the same interface.

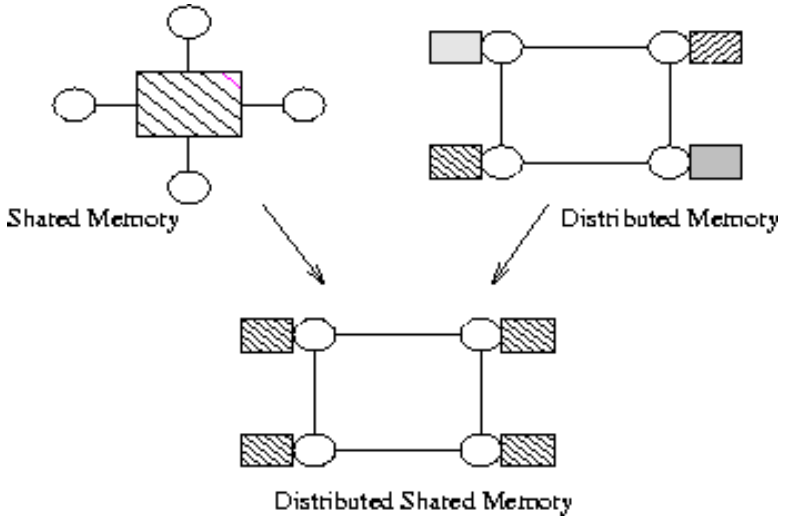
PRACTICAL - 9

AIM: Explain the concept of shared memory and distributed memory

Distributed Memory (DM)

- In DSM the different physical memories are logically shared over a large address space(virtual memory).
- So the processes going on accesses the physical memory through these logically shared address space.
- DSM have no physical shared memory.
- The shared memory model provides a virtual address space shared between all nodes.
- The term "shared" does not mean that there is a single centralized memory but "shared" essentially means that the address space is shared (same physical address on two processors refers to the same location in memory).

- A distributed shared memory is a mechanism allowing end-users' processes to access shared data without using inter-process communications. In other words, the goal of a DSM system is to make inter-process communications transparent to end-users. Both hardware and software implementations have been proposed in the literature. From a programming point of view, two approaches have been studied:
- **Shared virtual memory:** This notion is very similar to the well-known concept of paged virtual memory implemented in mono-processor systems. The basic idea is to group all distributed memories together into a single wide address space. Drawbacks: such systems do not allow to take into account the semantics of shared data: the data granularity is arbitrarily fixed to some page size whatever the type and the actual size of the shared data might be. The programmer has no means to provide informations about these data.
- **Object DSM:** in that class of approaches, shared data are objects i.e. variables with access functions. In his applications, the user has only to define which data (objects) are shared. The whole management of the shared objects (creation, access, modification) is handled by the DSM system. In opposite of SVM systems which work at operating system layer, objects DSM systems actually propose a programming model alternative to the classical message-passing.



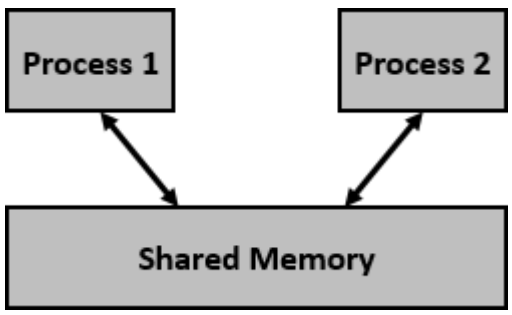
Shared Memory (SM)

Shared memory is a memory shared between two or more processes. However, why do we need to share memory or some other means of communication?

To reiterate, each process has its own address space, if any process wants to communicate with some information from its own address space to other processes, then it is only possible with IPC (inter process communication) techniques. As we are already aware, communication can be between related or unrelated processes.

Usually, inter-related process communication is performed using Pipes or Named Pipes. Unrelated processes (say one process running in one terminal and another process in another terminal) communication can be performed using Named Pipes or through popular IPC techniques of Shared Memory and Message Queues.

We have seen the IPC techniques of Pipes and Named pipes and now it is time to know the remaining IPC techniques viz., Shared Memory, Message Queues, Semaphores, Signals, and Memory Mapping.



We know that to communicate between two or more processes, we use shared memory but before using the shared memory what needs to be done with the system calls, let us see this −

- Create the shared memory segment or use an already created shared memory segment (shmget())

- Attach the process to the already created shared memory segment (shmat())
- Detach the process from the already attached shared memory segment (shmdt())
- Control operations on the shared memory segment (shmctl())

Shared Memory

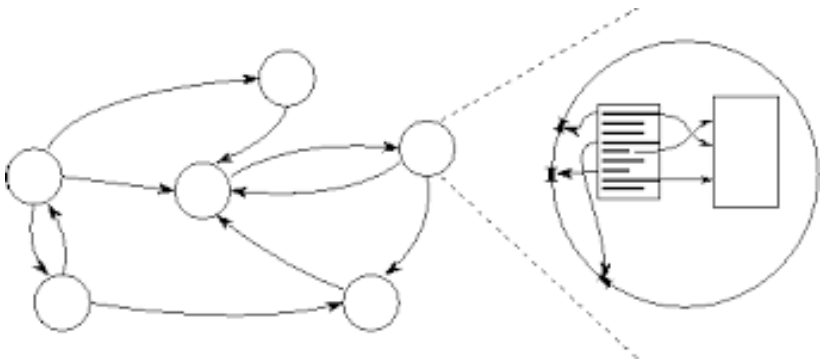
- In computer science, shared memory is memory that may be simultaneously accessed by multiple programs with an intent to provide communication among them or avoid redundant copies.
- Shared memory is an efficient means of passing data between programs.
- Depending on context, programs may run on a single processor or on multiple separate processors.
- In computer software, shared memory is either a method of inter-process communication (IPC), i.e. a way of exchanging data between programs running at the same time.
- One process will create an area in RAM which other processes can access; a method of conserving memory space by directing accesses to what would ordinarily be copies of a piece of data to a single instance instead, by using virtual memory mappings.

PRACTICAL - 10

AIM: Demonstrate the concept of parallel programming

Parallel Programming

In very simple terms, it is the use of multiple resources, in this case, processors, to solve a problem. This type of programming takes a problem, breaks it down into a series of smaller steps, delivers instructions, and processors execute the solutions at the same time. It is also a form of programming that offers the same results as concurrent programming but in less time and with more efficiency. Many computers, such as laptops and personal desktops, use this programming in their hardware to ensure that tasks are quickly completed in the background.



Advantages

There are two major advantages to using this programming over concurrent programming. One is that all processes are sped up when using parallel structures, increasing both the efficiency and resources used in order to achieve quick results. Another benefit is that parallel computing is more cost efficient than concurrent programming simply because it takes less time to get the same results. This is incredibly important, as parallel processes are necessary for accumulating massive amounts of data into data sets that can be easy to process or for solving complicated problems.

Disadvantages

There are several disadvantages to parallel processing. The first is that it can be difficult to learn; programming that targets parallel architectures can be overwhelming at first, so it does take time to fully understand. Additional, code tweaking is not straightforward and must be modified for different target architectures to properly improve performance. It's also hard to estimate consistent results because communication of results can be problematic for certain architectures. Finally, power consumption is a problem for those instituting a multitude of processors for various architectures; a variety of cooling technologies will be required in order cool the parallel clusters.

Where is it Used?

This type of programming can be used for everything from science and engineering to retail and research. It's most common use from a societal perspective is in web search engines, applications, and multimedia technologies. Nearly every field in America uses this programming in some aspect, whether for research and development or for selling their wares on the web, making it an important part of computer science.

Classification of parallel programming models

Classifications of parallel programming models can be divided broadly into two areas: process interaction and problem decomposition.

PROCESS INTERACTION

Process interaction relates to the mechanisms by which parallel processes are able to communicate with each other. The most common forms of interaction are shared memory and message passing, but interaction can also be implicit (invisible to the programmer).

Shared memory

Shared memory is an efficient means of passing data between processes. In a shared-memory model, parallel processes share a global address space that they read and write to asynchronously. Asynchronous concurrent access can lead to race conditions, and mechanisms such as locks, semaphores and monitors can be used to avoid these. Conventional multi-core processors directly support shared memory, which many parallel programming languages and libraries, such as Cilk, OpenMP and Threading Building Blocks, are designed to exploit.

Message passing

In a message-passing model, parallel processes exchange data through passing messages to one another. These communications can be asynchronous, where a message can be sent before the receiver is ready, or synchronous, where the receiver must be ready. The Communicating sequential processes (CSP) formalisation of message passing uses synchronous communication channels to connect processes, and led to important languages such as Occam, Limbo and Go. In contrast, the actor model uses asynchronous message passing and has been employed in the design of languages such as D, Scala and SALSA.

Implicit interaction

In an implicit model, no process interaction is visible to the programmer and instead the compiler and/or runtime is responsible for performing it. Two examples of implicit parallelism are with domain-specific languages where the concurrency within high-level operations is prescribed, and with functional programming languages because the absence of side-effects allows non-dependent functions to be executed in parallel. However, this kind of parallelism is difficult to manage[7] and functional languages such as Concurrent Haskell and Concurrent ML provide features to manage parallelism explicitly and correctly.

PROBLEM DECOMPOSITION

A parallel program is composed of simultaneously executing processes. Problem decomposition relates to the way in which the constituent processes are formulated.

Task parallelism

A task-parallel model focuses on processes, or threads of execution. These processes will often be behaviourally distinct, which emphasises the need for communication. Task parallelism is a natural way to express message-passing communication. In Flynn's taxonomy, task parallelism is usually classified as MIMD/MPMD or MISD.

Data parallelism

A data-parallel model focuses on performing operations on a data set, typically a regularly structured array. A set of tasks will operate on this data, but independently on disjoint partitions. In Flynn's taxonomy, data parallelism is usually classified as MIMD/SPMD or SIMD.

Implicit parallelism

As with implicit process interaction, an implicit model of parallelism reveals nothing to the programmer as the compiler, the runtime or the hardware is responsible. For example, in compilers, automatic parallelization is the process of converting sequential code into parallel code, and in computer architecture, superscalar execution is a mechanism whereby instruction-level parallelism is exploited to perform operations in parallel.